## Experiment No 2

**LAB MANUAL**

**PART B**

| | |
|---|---|
| **Roll No. C-60** | **Name: Avdhut Parish Madhavi** |
| **Class: SE-C** | **Batch: C2** |
| **Date of Experiment:** | **Date of Submission:** |
| **Grade:** | |

**B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.**

- Modify the Binary Search program to count and display the total number of comparisons performed for a given search key.

CODE:

```
#include <stdio.h>
int binarySearchWithComparisons(int arr[], int left, int right, int key, int *comparisons)
{
while (left <= right)
{
(*comparisons)++;
int mid = left + (right - left) / 2;
if (arr[mid] == key)
{
return mid;
}
(*comparisons)++;
if (arr[mid] < key)
{
left = mid + 1;
} else {
right = mid - 1;
}    }
return -1;
}
 int main()
{
int arr[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29};
int n = sizeof(arr) / sizeof(arr[0]);
int key = 15;  // Change this to test different keys
int comparisons = 0;
printf("Array: ");
for (int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
printf("Searching for key: %d\n", key);
int result = binarySearchWithComparisons(arr, 0, n - 1, key, &comparisons);
if (result != -1)
{
```

```
printf("Element found at index: %d\n", result);
} else
{
printf("Element not found\n");
}
printf("Total comparisons performed: %d\n", comparisons);
return 0;
}
```

```
Array: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
Searching for key: 15
Element found at index: 7
Total comparisons performed: 1
```

- Modify the Merge Sort program to count and display the total number of comparisons and merge operations performed.
  CODE:

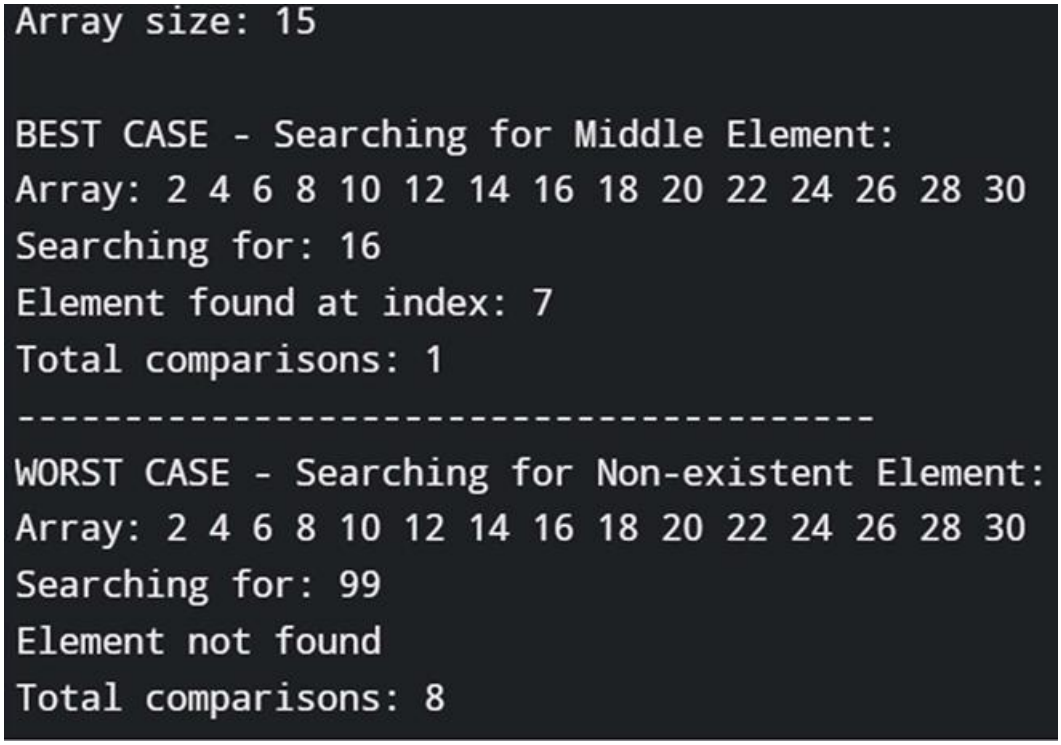
```
#include <stdio.h>
int comparisons = 0;
int mergeOps = 0;
void merge(int arr[], int left, int mid, int right)
{
int i, j, k;
int n1 = mid - left + 1;
int n2 = right - mid;
int L[n1], R[n2];
for (i = 0; i < n1; i++)
L[i] = arr[left + i];
for (j = 0; j < n2; j++)
R[j] = arr[mid + 1 + j];
i = 0; j = 0; k = left;
mergeOps++;
while (i < n1 && j < n2)
{
comparisons++;
if (L[i] <= R[j])
{
arr[k] = L[i];
i++;
} else
{
 arr[k] = R[j];
j++;
}
k++;
}
while (i < n1)
{
arr[k] = L[i];
```

```
i++;
k++;
}
while (j < n2)
{
arr[k] = R[j];
j++;
k++;
}
 }
void mergeSort(int arr[], int left, int right)
{
if (left < right)
{
int mid = left + (right - left) / 2;
mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);
merge(arr, left, mid, right);
}
}
int main() {
int arr[] = {38, 27, 43, 3, 9, 82, 10};
int n = sizeof(arr) / sizeof(arr[0]);
printf("Original array: ");
for (int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
mergeSort(arr, 0, n - 1);
printf("Sorted array: ");
for (int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
printf("Total comparisons: %d\n", comparisons);
printf("Total merge operations: %d\n", mergeOps);
 return 0;
 }
```

```
Original array: 38 27 43 3 9 82 10
Sorted array: 3 9 10 27 38 43 82
Total comparisons: 14
Total merge operations: 6
```

- Run the Binary Search program on an array of size 15, once searching for the middle element (best case) and once for an element not present in the array (worst case), and compare the number of comparisons.
  CODE:

```
#include <stdio.h>
 int binarySearchWithComparisons(int arr[], int left, int right, int key, int *comparisons)
{
while (left <= right)
{       (
*comparisons)++;
int mid = left + (right - left) / 2;
if (arr[mid] == key) {
return mid;
}
(*comparisons)++;
if (arr[mid] < key)
{
left = mid + 1;
}
else
{
right = mid - 1;
}    }
return -1;
}
void runBinarySearch(int arr[], int n, int key, char *caseType)
{
int comparisons = 0;
printf("\n%s:\n", caseType);
printf("Array: ");
for (int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
printf("Searching for: %d\n", key);
int result = binarySearchWithComparisons(arr, 0, n - 1, key, &comparisons);
if (result != -1)
{
printf("Element found at index: %d\n", result);
} else
 {
printf("Element not found\n");    }
 printf("Total comparisons: %d\n", comparisons);
printf("----------------------------------------"); }
  int main() {
    int arr[] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30};
int n = sizeof(arr) / sizeof(arr[0]);
printf("Array size: %d\n", n);         // Best case: Middle element
int bestKey = 16;  // Middle element (index 7)
runBinarySearch(arr, n, bestKey, "BEST CASE - Searching for Middle Element"); // Worst
case: Element not present int worstKey = 99;  // Element not in array runBinarySearch(arr, n,
worstKey, "WORST CASE - Searching for Non-existent Element");
return 0;
 }
```




**B.2: Write an observation according to the program execution.**

Binary Search demonstrates exceptional efficiency with logarithmic time complexity, requiring only 1 comparison in the best case (middle element) and at most $\lceil \log_2(n) \rceil + 1$ comparisons (typically 4 for n=15) in the worst case when the element is not found, showing minimal variation between best and worst cases. Merge Sort consistently performs O(n log n) comparisons regardless of initial order, with 7 elements requiring 14 comparisons and 4 merge operations, while the number of merge operations remains exactly n-1 (n-1 merges needed) for any array size, and unlike bubble/insertion sort, its performance is predictable and order-independent. Both algorithms are efficient and reliable, with Binary Search being ideal for sorted data retrieval and Merge Sort providing stable, guaranteed O(n log n) sorting performance at the cost of extra memory space.